Users

student pw:student
developer pw:developer
admin pw:redhatocp
root pw:redhat
Logging ing

OC LOGIN : *oc login -u developer -p developer https://api.ocp4.example.com:6443*

PODMAN LOGIN: *podman login registry.redhat.io*

To log out from all at once: *podman logout --all*

Podman Basics

Fetching container image, pulling container image
podman pull registry.access.redhat.com/ubi9/ubi-minimal:9.5

Listing images, listing container images
podman images

Running a container image,
podman run registry.access.redhat.com/ubi9/ubi-minimal:9.5 echo 'Red Hat'
*** add --rm to automatically remove after running** *(podman run --rm IMAGE)*
*** add --name NAME to assign it a name** *(podman run --name NAME IMAGE)*
*** add -p to map a port from your computer to the container** *(podman run -p 8080:8080 IMAGE)*
:to a specific host (-p IPADRESS:8080:80)
*** add -d to make the container not block the terminal** *(podman run -b -p 8080:8080 IMAGE)*
***add -e to add environmental variables** *(podman run -e NAME='Value' IMAGE)*
***add --net to connect it to a network** *(podman run -d --name CONNAME --net NETNAME1,NETNAME2)*
add --add-hostto add a host to the container (**podman run --add-host=HOSTNAME:IPADDRESS IMAGE**)
add --secret to run it with a secret (**podman run --secret SECRETNAME NAME**)
add** --volume MACHINEDIRECTORY:CONTAINERDIRECTORY **to mount data
add extra linux commands at the end of the line

Ports checking:

List port mapping for a container: *podman port INSTANCENAME*

***add --all to list ports for all containers** *(podman port --all)*

Processes in containers:
start processes in containers: *podman exec CONTAINER*
***add -e to add environmental variables** *(podman exec -e NAME='Value' CONTAINER)*
add --interactive to accept user input
add --tty to allocate a pseudo terminal
add --latest to execute the command in the last created container

Copying files in and out of containers

```
podman cp [options] [container:]source_path [container:]destination_path
```

Example:
podman cp a3bd6c81092e:/tmp/logs .

Inspect containers
podman inspect CONTAINER_NAME
****to get specific json field:** *podman inspect --format='{{.JSON.FIELD}}' CONTAINER_NAME*

Extra linux commands
printing something in the terminal: *echo STRING*
printing environmental variables: *printenv NAME*
printing the contents of a file: *cat path/to/file*
edit the contents of a file: *nano* path/to/file

List running containers
podman ps
*** **add --all to show running and stopped containers** (podman ps --all)*
*** **add --format=json to get the UUID long Container ID** (podman ps --all --format=json)*

Stopping Containers
podman stop CONTAINER_NAME
****add --all to stop all containers:** *podman stop --all*
****add --time=100 to stop them after 10 seconds:** *podman stop --time=100*

Force kill them:
podman kill CONTAINER_NAME

To restart a container: *podman restart CONTAINER_NAME*

To remove a container: *podman rm CONTAINER_NAME*
*****add --force to forcefully remove it:** *podman rm CONTAINER_NAME --force*
*****use --all to remove all non-running containers:** podman rm --all
****combine to remove all:** *podman rm --all --force*

Pausing containers
podman pause CONTAINER_NAME
podman unpause CONTAINER_NAME

Export and Import Containers

To save an image: *podman save --output FILENAME.TAR IMAGE_REGISTRY*

To load an image: *podman load --input FILENAME.TAR*

Container networking

Create a podman network: *podman network create NAME*

to block traffic from other networks add: *podman network create -o isolate=TRUE NAME*

to

List existing networks: *podman network ls*

Check configuration data for the network: *podman network inspect NAME*

Remove a network: *podman network rm NAME*

Remove all networks not being used by containers: *podman network prune*

Connect running container to existing network: *podman network connect*

Disconnect a container from a network: *podman network disconnect*

Get private IP address of the container within a network:

podman inspect applicationName --f '{{.NetworkSettings.Networkspodman0.0networkname.IPAddress}}'

Container images

Pulling an image: *podman pull REGISTRY_IMAGE_URL*

Attach tags to images: *podman image tag LOCAL_IMAGE_NAME:TAG*

List images: *podman image ls*

Building Images

To build an image: *podman build --file CONTAINERFILE --tag IMAGETAG*

****add --squash-all to squash the layers:*** *podman build --squash-all IMAGETAG*

Pushing an image: *podman push IMAGETAG quay.io/QUAYUSERNAME/IMAGENAME:TAG*

Inspecting images: *podman image inspect IMAGE_REGISTRY_URL*

****add** --format="{{JSON.PATH}}"* ***to inspect specific property***

Removing images: *podman image rm REGISTRY/NAMESPACE/IMAGE_NAME:TAG*

****add -f to forcefully remove it**: podman image rm IMAGE*

****add --all to remove all images:*** *podman image rm --all*

Delete dangling images: *podman image prune*

****add -a to delete both dangling and unused:*** podman image prune -a

***add -f to force their deletion:** *podman image prune -af*

Manage registries with Skopeo

To read image metadata: *skopeo inspect REGISTRY_IMAGE_URL*

Copy an image to a registry: *skopeo copy REGISTRY_IMAGE_URL DESTINATION_REPOSITORY_URL*

Podman Secrets

To create a podman secret: *echo "SECRET TEXT" > SECRETFILENAME*

podman secret create SECRETNAME SECRETFILENAME

To combine them:

```
[user@host ~]$ printf "R3d4ht123" | podman secret create dbsecret2 -
```

***add --driver to specify the driver:** *podman secret create SECRETNAME --driver pass*

To delete a secret: *podman secret rm SECRETNAME*

Storing data with volumes

To create a volume: *podman volume create VOLUME_NAME*

To inspect the volume: *podman volume inspect VOLUME_NAME*

To import a volume: *podman volume import VOLUME_NAME ARCHIVE_NAME*

To export a volume: *podman volume export VOLUME_NAME --output ARCHIVE_NAME*

Loading data with a database client: *podman cp SQL_FILE TARGET_DB_CONTAINER:CONTAINER_PATH*

Export database data: *podman exec POSTGRESQL_CONTAINER pg_dump -Fc DATABASE -f BACKUP_DUMP*

Logging

To check the logs of a container: *podman logs CONTAINER*

The Compose File

The Compose file is a YAML file that contains the following sections:
version (deprecated): Specifies the Compose version used.
services: Defines the containers used.
networks: Defines the networks used by the containers.
volumes: Specifies the volumes used by the containers.
configs: Specifies the configurations used by the containers.
secrets: Defines the secrets used by the containers.

```
name: compose-lab


services:
  wiremock:
    container_name: "quotes-provider"
    image: "registry.ocp4.example.com:8443/redhattraining/wiremock"
    volumes:
      - ~/DO188/labs/compose-lab/wiremock/stubs:/home/wiremock:Z
```

```
    networks:
      - backend-net
  quotes-api:
    container_name: "quotes-api"
    image: "registry.ocp4.example.com:8443/redhattraining/podman-quotesapi-compose"
    networks:
      - backend-net
      - frontend-net
    environment:
      QUOTES_SERVICE: "http://quotes-provider:8080"
  quotes-ui:
    container_name: "quotes-ui"
    image: "registry.ocp4.example.com:8443/redhattraining/podman-quotes-ui"
    networks:
    - frontend-net
    ports:
      - "3000:8080"

networks:
  backend-net: {}
  frontend-net: {}
```

Use *podman-compose up* ***to execute a compose file***
****add** *-d* **to start it in the background**
****add** *--force-recreate* **to recreate it on start**
****add** *--remove-orphans* **to remove containers that do not correspond to the ones in the compose file**

Use *podman-compose down* **to stop and remove containers defined as services**
****add** *-v* **to remove anonymous volumes when recreating**

Use *podman-compose stop* **to stop the containers that are defined in the file**

Container Files

Command	Function
FROM	Sets the base image for the resulting container image. Takes the name of the base image as an argument.
WORKDIR	Sets the current working directory for commands that run inside the container. Instructions that

	follow the `WORKDIR` instruction run within this directory.
`COPY` and `ADD`	Copy files from the build host into the file system of the resulting container image. Relative paths use the host current working directory, known as the build context. Both instructions use the working directory within the container as defined by the WORKDIR instruction. The ADD instruction adds the following functionality: Copying files from URLs. Unpacking tar archives in the destination image. Because the ADD instruction adds functionality that might not be obvious, developers tend to prefer the COPY instruction for copying local files into the container image.
`RUN`	Runs a command in the container and commits the resulting state of the container to a new layer within the image.
`ENTRYPOINT`	Sets the executable to run when the container is started.
`CMD`	Runs a command when the container is started. This command is passed to the executable defined by ENTRYPOINT. Base images define a default ENTRYPOINT, which is usually a shell executable, such as Bash.
`USER`	Changes the active user within the container. Instructions that follow the `USER` instruction run as this user, including the `CMD` instruction. It is a good practice to define a different user other than `root` for security reasons.
`LABEL`	Adds a key-value pair to the metadata of the image for organization and image selection.
`EXPOSE`	Adds a port to the image metadata indicating that an application within the container binds to this port. This instruction does not bind the port on the host and is for documentation purposes.
`ENV`	Defines environment variables that are available in the container. You can declare multiple `ENV` instructions within the Containerfile. You can use the `env` command inside the container to view each of the environment variables.

ARG	Defines build-time variables, typically to make a customizable container build. Developers commonly configure the `ENV` instructions by using the `ARG` instruction. This is useful for preserving the build-time variables for runtime.
VOLUME	Takes a path argument. Marks the path as a volume mount point and, when you create a container, Podman creates a volume for the path. You can define more than one path to create multiple volumes.
#coments	Commenting

EXAMPLE code

Use the private IP address to make a request to the `/times` API endpoint from another container. Run the `curl` command inside a `ubi-minimal` container to fetch the current time for Bangkok, which has the code `BKK`. Attach the container to the `cities` network.

```
podman run --rm --network cities registry.ocp4.example.com:8443/ubi9/ubi-minimal:9.5 curl -s
http://DNS_NAME:8080/times/BKK && echo
```

The following command sets the `ENVIRONMENT` variable and then executes the `env` command to print all environment variables. In this example, the container name is not necessary because the `-l` option is used

```
[user@host ~]$ podman exec -e ENVIRONMENT=dev -l env
```

Use the following command to copy the `nginx.conf` file from the `nginx-test` container to the `nginx-proxy` container:

```
[user@host ~]$ podman cp nginx-test:/etc/nginx/nginx.conf nginx-proxy:/etc/nginx
```

Use the following command to copy the `nginx.conf` file from the `nginx-test` container to the `nginx-proxy` container:

```
[user@host ~]$ podman cp nginx-test:/etc/nginx/nginx.conf nginx-proxy:/etc/nginx
```

Reload the server configuration.

```
[student@workstation ~]$ podman exec nginx nginx -s reload
```

In the following command, the `Status` field of the `State` object is retrieved for a container called `redhat`.

```
[user@host ~]$ podman inspect --format='{{.State.Status}}' redhat
```

To enable a Quadlet systemd unit for the current user run:

```
[user@host ~]$ systemctl --user daemon-reload
```

To manage a user Quadlet service, use the `systemctl` command. You can replace *<action>* in the following command with `start`, `stop`, or `status`, to control the service state.

```
[user@host ~]$ systemctl --user <action> <serviceName>.service
```

When you use the `--user` option, systemd starts the service when you log in, and stops it when you log out. You can avoid stopping your service when you log out by running the `loginctl enable-linger` command.

```
[user@host ~]$ loginctl enable-linger
```

To revert the operation, use the `loginctl disable-linger` command.

Podman stores the credentials in the `${XDG_RUNTIME_DIR}/containers/auth.json` file, where the `${XDG_RUNTIME_DIR}` refers to a directory specific to the current user. The credentials are encoded in the `base64` format:

```
[user@host ~]$ cat ${XDG_RUNTIME_DIR}/containers/auth.json
{
        "auths": {
                "registry.redhat.io": {
                        "auth": "dXNlcjpodW50ZXIy"
                }
        }
```

```
}
[user@host ~]$ echo -n dXNlcjpodW50ZXIy | base64 -d
user:hunter2
```

Log in to RHOCP as the `admin` user.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
    https://api.ocp4.example.com:6443
```

Log in to the RHOCP registry with Podman.

```
[student@workstation ~]$ podman login -u $(oc whoami) -p $(oc whoami -t) \
  default-route-openshift-image-registry.apps.ocp4.example.com
```

Log in to the `registry.ocp4.example.com:8443` registry with Podman.

```
[student@workstation ~]$ podman login -u developer -p developer \
  registry.ocp4.example.com:8443
```

Add private registries to the default search list by including them in the `unqualified-search-registries` list in the `~/.config/containers/registries.conf` file. The following snippet shows the content of the `registries.conf` file:

```
unqualified-search-registries = ['registry.example.com:8443', 'registry.redhat.io', 'docker.io']
```

Verify that `python -m http.server` is the default command for this image. This Python command comes from the `CMD` instruction in the exercise Containerfile.

```
[student@workstation images-managing]$ podman image inspect simple-server \
  --format="{{.Config.Cmd}}"
[/bin/sh -c python -m http.server]
```

Make a request to the running container by using the `curl` command.

```
[student@workstation hello-server]$ curl http://localhost:3000/greet ;echo
{"hello":"world"}
```

To pass a Podman secret as an environmental variable, you must add `type=env` and `target=<ENV_VAR_NAME>` subparameters to the `--secret` parameter.

The previous example with environmental variables would be as follows:

```
[user@host ~]$ podman run -it --secret dbsecret,type=env,target=DB_SECRET \
--name myapp registry.access.redhat.com/ubi8/ubi /bin/bash
[root@f9fedddbc81a /]# printenv DB_SECRET
R3d4ht123
```

The following example implements the two-stage build process.

```
# First stage

FROM registry.access.redhat.com/ubi8/nodejs-14:1 as builder
COPY ./ /opt/app-root/src/
RUN npm install

RUN npm run build

# Second stage

FROM registry.access.redhat.com/ubi8/nginx-120

COPY --from=builder /opt/app-root/src/ /usr/share/nginx/html
```

Determine the current user by running the `id` command:

```
[user@host ~]$ podman run registry.access.redhat.com/ubi9/ubi id
uid=0(root) gid=0(root) groups=0(root)
```

Some applications cannot use the default COW file system in a specific directory for performance reasons, but use persistence or data sharing for that directory.

For such cases, you can use the `tmpfs` mount type, which means that the data in a mount is ephemeral but does not use the COW file system:

```
[user@host ~]$ podman run -e POSTGRESQL_ADMIN_PASSWORD=redhat --network lab-net \
  --mount  type=tmpfs,tmpfs-size=512M,destination=/var/lib/pgsql/data \
  registry.redhat.io/rhel9/postgresql-13:1
```

For example, the following command creates an ephemeral container to load data into a PostgreSQL database:

```
[user@host ~] podman run -it --rm \
  -e PGPASSWORD=DATABASE_PASSWORD \
  -v ./SQL_FILE:/tmp/SQL_FILE:Z \
  --network DATABASE_NETWORK \
  registry.redhat.io/rhel8/postgresql-12:1-113 \
  psql -U DATABASE_USER -h DATABASE_CONTAINER \
       -d DATABASE_NAME -f /tmp/SQL_FILE
```

You can use the `podman port` *`CONTAINER`* command to list the current container port mapping.

```
[user@host ~]$ podman port CONTAINER
8000/tcp -> 0.0.0.0:8080
```

To verify the application ports in use, list the open network ports in the running container. Use Linux commands such as the socket statistics (`ss`) command to list open ports. A socket is the

combination of a port and an IP address. The `ss` command lists the open sockets in a system. You can provide the `ss` command with options to filter and produce the desired output:

- `-p`: display the process using the socket
- `-a`: display listening and established connections
- `-n`: display numeric ports instead of mapped service names
- `-t`: display TCP sockets

```
[user@host ~]$ podman exec CONTAINER ss -pant
Netid State  ... Local Address:Port   Peer Address:Port   Process
tcp   LISTEN ...       0.0.0.0:9091        0.0.0.0:*      users:("python",pid=...)
...output omitted...
```

Use the `nsenter` host command to run the host `ss` command in the container.

Copy the container process ID (PID) for later use with the `nsenter` command. The PID number might differ for your container.

```
[student@workstation ~]$ podman inspect smart-home-api --format '{{.State.Pid}}'
2809
```

Run the `nsenter` command. Use the `-t` option to set the container PID obtained previously as the target. Use the `-n` option to enter the `smart-home-pid` network namespace.

```
[student@workstation ~]$ sudo nsenter -t 2809 -n ss -pant
Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port Process
tcp   LISTEN 0      2048       0.0.0.0:8000       0.0.0.0:*  users: "uvicorn",pid=76641,fd=23
...output omitted...
```

The application server `uvicorn` is listening on the `0.0.0.0` address on port 8000. In the preceding steps, you attempted to test the server on port 8080, which is incorrect.

```
services:
  db-admin:
    image: "registry.ocp4.example.com:8443/crunchydata/crunchy-pgadmin4:ubi8-4.30-1"
    container_name: "compose_environments_pgadmin"
    environment:
      PGADMIN_SETUP_EMAIL: user@example.com
      PGADMIN_SETUP_PASSWORD: redhat
```

```
    ports:
      - "5050:5050"
  db:
    image: "registry.ocp4.example.com:8443/rhel9/postgresql-13:1"
    container_name: "compose_environments_postgresql"
    environment:
      POSTGRESQL_USER: backend
      POSTGRESQL_DATABASE: rpi-store
      POSTGRESQL_PASSWORD: redhat
    ports:
      - "5432:5432"
    volumes:
      - ./database_scripts:/opt/app-root/src/postgresql-start:Z
      - rpi:/var/lib/pgsql/data

volumes:
  rpi: {}

networks:
  app-net: {}
  db-net: {}
```

```
ARG VERSION=1.0.0
FROM registry.ocp4.example.com:8443/ubi8/openjdk-17:1.12 AS builder
LABEL maintainer="student@example.com"
USER root
WORKDIR /build
COPY --chown=jboss:jboss . .
ADD https://example.com/config.tar.gz ./
RUN mvn package -Dversion=${VERSION}
ENV APP_HOME=/opt/app

FROM registry.ocp4.example.com:8443/ubi8/openjdk-17-runtime:1.12
WORKDIR /opt/app
```

```
COPY --from=builder /build/target/app.jar .

EXPOSE 8080

VOLUME /opt/app/data

ENV JAVA_OPTS="-Xmx512m"

ENTRYPOINT ["java", "-jar"]

CMD ["app.jar"]
```